Labsheet 1 : SECS05-AS01
Mastery in Data Analytics and Visualizations and Career Advancement

Khushi Choudhary CU240251564

In [2]:
```
# Khushi Choudhary CU240251564
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
```

In [3]:
```
df = pd.read_csv('train.csv') # Load the dataset
print(df.head()) # Inspect the first 5 rows
print(df.info()) # Check data types and non-null counts
```

```
   PassengerId  Pclass                                          Name     Sex  \
0          892       3                              Kelly, Mr. James    male
1          893       3              Wilkes, Mrs. James (Ellen Needs)  female
2          894       2                     Myles, Mr. Thomas Francis    male
3          895       3                              Wirz, Mr. Albert    male
4          896       3  Hirvonen, Mrs. Alexander (Helga E Lindqvist)  female

    Age  SibSp  Parch   Ticket     Fare Cabin Embarked
0  34.5      0      0   330911   7.8292   NaN        Q
1  47.0      1      0   363272   7.0000   NaN        S
2  62.0      0      0   240276   9.6875   NaN        Q
3  27.0      0      0   315154   8.6625   NaN        S
4  22.0      1      1  3101298  12.2875   NaN        S
<class 'pandas.DataFrame'>
RangeIndex: 418 entries, 0 to 417
Data columns (total 11 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  418 non-null    int64
 1   Pclass       418 non-null    int64
 2   Name         418 non-null    str
 3   Sex          418 non-null    str
 4   Age          332 non-null    float64
 5   SibSp        418 non-null    int64
 6   Parch        418 non-null    int64
 7   Ticket       418 non-null    str
 8   Fare         417 non-null    float64
 9   Cabin        91 non-null     str
 10  Embarked     418 non-null    str
dtypes: float64(2), int64(4), str(5)
memory usage: 36.1 KB
None
```

In [4]:
```
print(df.describe()) # View statistical summary of numerical columns
```

```
       PassengerId      Pclass         Age       SibSp       Parch        Fare
count   418.000000  418.000000  332.000000  418.000000  418.000000  417.000000
mean   1100.500000    2.265550   30.272590    0.447368    0.392344   35.627188
std     120.810458    0.841838   14.181209    0.896760    0.981429   55.907576
min     892.000000    1.000000    0.170000    0.000000    0.000000    0.000000
25%     996.250000    1.000000   21.000000    0.000000    0.000000    7.895800
50%    1100.500000    3.000000   27.000000    0.000000    0.000000   14.454200
75%    1204.750000    3.000000   39.000000    1.000000    0.000000   31.500000
max    1309.000000    3.000000   76.000000    8.000000    9.000000  512.329200
```

In [8]:
```python
# Check for missing values
print(df.isnull().sum())

# 1. Fill 'Age' missing values with the median
df['Age'] = df['Age'].fillna(df['Age'].median())
# 2. Fill 'Embarked' missing values with the most frequent value (mode)
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
# 3. Drop 'Cabin' column as it has too many missing values (>75%)
df.drop('Cabin', axis=1, inplace=True)

# Verify missing values
print(df.isnull().sum())
```

```
PassengerId      0
Pclass           0
Name             0
Sex              0
Age             86
SibSp            0
Parch            0
Ticket           0
Fare             1
Cabin          327
Embarked         0
dtype: int64
PassengerId    0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           1
Embarked       0
dtype: int64
```

In [9]:
```python
# Check number of duplicates
print("Duplicates before:", df.duplicated().sum())
# Drop duplicate rows
df = df.drop_duplicates()
print("Duplicates after:", df.duplicated().sum())
```

```
Duplicates before: 0
Duplicates after: 0
```

In [ ]:
```python
# Label Encoding
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
# One-Hot Encoding
# drop_first=True prevents the dummy variable trap
```

```
df = pd.get_dummies(df, columns=['Embarked'], drop_first=True)
print(df.head())
```

```
   PassengerId  Pclass                                          Name  Sex  \
0          892       3                              Kelly, Mr. James    0
1          893       3              Wilkes, Mrs. James (Ellen Needs)    1
2          894       2                     Myles, Mr. Thomas Francis    0
3          895       3                              Wirz, Mr. Albert    0
4          896       3  Hirvonen, Mrs. Alexander (Helga E Lindqvist)    1

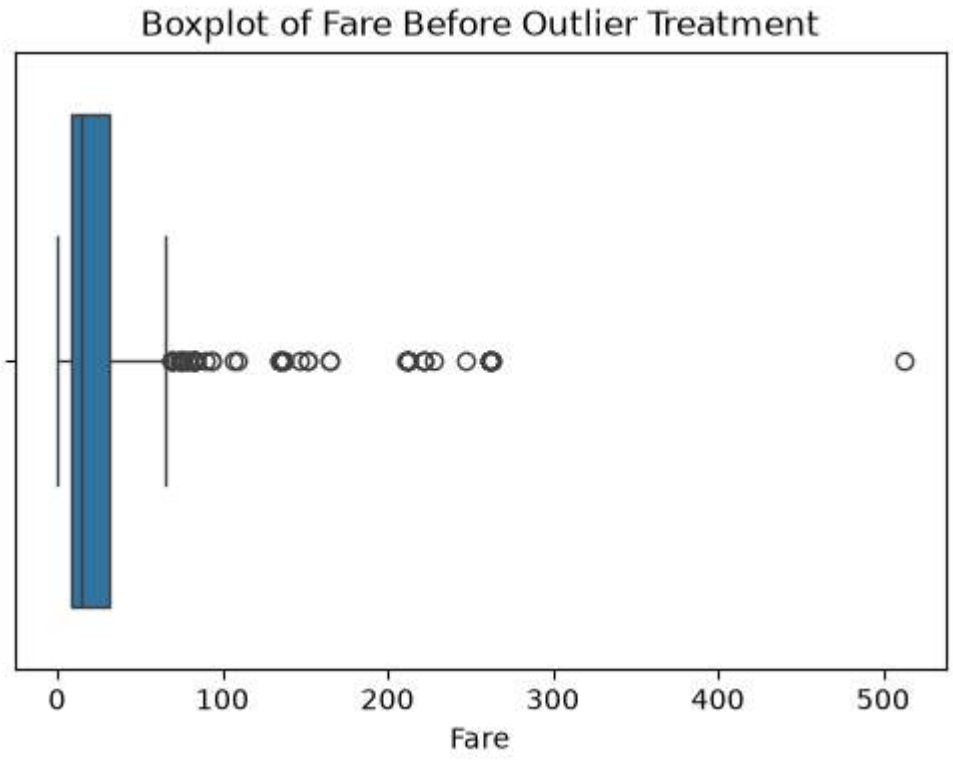
    Age  SibSp  Parch   Ticket     Fare  Embarked_Q  Embarked_S
0  34.5      0      0   330911   7.8292        True       False
1  47.0      1      0   363272   7.0000       False        True
2  62.0      0      0   240276   9.6875        True       False
3  27.0      0      0   315154   8.6625       False        True
4  22.0      1      1  3101298  12.2875       False        True
```

In [11]:
```
# Visualize outliers in 'Fare' using a Box Plot
plt.figure(figsize=(6, 4))
sns.boxplot(x=df['Fare'])
plt.title('Boxplot of Fare Before Outlier Treatment')
plt.show()

# Calculate Q1, Q3, and IQR for 'Fare'
Q1 = df['Fare'].quantile(0.25)
Q3 = df['Fare'].quantile(0.75)
IQR = Q3 - Q1

# Define upper and lower bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Treat outliers by capping (replacing outliers with the boundary limits)
df['Fare'] = np.where(df['Fare'] > upper_bound, upper_bound, df['Fare'])
df['Fare'] = np.where(df['Fare'] < lower_bound, lower_bound, df['Fare'])
```



In [12]:
```
# Initialize the StandardScaler
scaler = StandardScaler()
# Standardize 'Age' and 'Fare' to have mean=0 and variance=1
df[['Age', 'Fare']] = scaler.fit_transform(df[['Age', 'Fare']])
print(df[['Age', 'Fare']].head())
```

```
        Age      Fare
0  0.386231 -0.794535
1  1.371370 -0.833638
2  2.553537 -0.706902
3 -0.204852 -0.755238
4 -0.598908 -0.584292
```

In [13]:
```
# Perform feature engineering
# Create a new feature 'Family_Size'
df['Family_Size'] = df['SibSp'] + df['Parch'] + 1
# Drop the original columns as they are now captured in 'Family_Size'
df.drop(['SibSp', 'Parch'], axis=1, inplace=True)
# Drop irrelevant columns
df.drop(['PassengerId', 'Name', 'Ticket'], axis=1, inplace=True)
print(df.head())
```

```
   Pclass  Sex       Age      Fare  Embarked_Q  Embarked_S  Family_Size
0       3    0  0.386231 -0.794535        True       False            1
1       3    1  1.371370 -0.833638       False        True            2
2       2    0  2.553537 -0.706902        True       False            1
3       3    0 -0.204852 -0.755238       False        True            1
4       3    1 -0.598908 -0.584292       False        True            3
```

In [ ]:
```
# Khushi Choudhary CU240251564
# Save the preprocessed DataFrame to a new CSV file
df.to_csv('cleaned_titanic_dataset.csv', index=False)
print("Dataset cleaned and saved successfully as 'cleaned_titanic_dataset.csv'!")
```

```
Dataset cleaned and saved successfully as 'cleaned_titanic_dataset.csv'!
```